

Relatório Técnico do Projeto da Unidade Curricular de Fundamentos de Desenvolvimento de Software

Aplicação: Pesca Tudo

Instituto Superior de Engenharia do Porto (ISEP)

Gonçalo Maia 1221923, Raul Choupina 1230593, Fernando Procópio 1210874

Grupo: 2
Turma: IDC

Resumo: Este relatório serve como suporte escrito para o projeto de desenvolvimento de software utilizando a linguagem de programação orientada a objetos C++. O objetivo deste relatório é, por meio do uso de um padrão de arquitetura de software, esclarecer as decisões relacionadas à implementação do projeto e especificar suas funcionalidades por meio de diagramas UML. A elaboração desse tipo de relatório é importante para garantir que todas as partes interessadas envolvidas no projeto estejam em sintonia durante a implementação e a gestão do software.

1 Problema e Abordagem

As lojas de artigos de pesca necessitam de uma forma eficiente de gerir produtos, clientes, fornecedores e encomendas. A realização destas tarefas de forma manual pode originar dificuldades no controlo da informação, atrasos no processamento de pedidos e uma maior probabilidade de ocorrência de erros operacionais.

Para responder a esta necessidade foi desenvolvida a aplicação PescaTudo, com o intuito de administrar produtos, clientes, fornecedores e encomendas através de diferentes perfis de utilizador. Apresenta três tipos principais de utilizadores: cliente, fornecedor e gerente.

Os clientes sem sessão iniciada podem consultar os produtos disponíveis na loja sem necessidade de autenticação. No entanto, para adicionar produtos ao carrinho, efetuar encomendas, consultar o histórico de compras e avaliar produtos, é necessário criar uma conta e iniciar sessão na aplicação.

Os fornecedores dispõem de funcionalidades relacionadas com os produtos que fornecem, podendo consultar a lista de produtos associados, visualizar os respetivos detalhes, atualizar a disponibilidade dos produtos e responder a pedidos de reposição enviados pelo gerente.

Por sua vez, o gerente assume a responsabilidade pela administração global do sistema, sendo responsável pela gestão de produtos, clientes e fornecedores, bem como pela consulta e acompanhamento das encomendas efetuadas pelos clientes e dos pedidos realizados aos fornecedores.

Após a fase de análise e design desenvolvida nas iterações anteriores, esta terceira iteração centra-se na implementação da solução proposta e na validação das funcionalidades desenvolvidas. Assim, o presente relatório apresenta a estrutura final da aplicação, as principais decisões de implementação adotadas, os componentes desenvolvidos e os testes realizados para verificar o correto funcionamento do sistema.

2 Enquadramento da Solução Desenvolvida

Nesta secção é apresentada uma visão geral da solução desenvolvida para o sistema PescaTudo. São descritos os principais artefactos produzidos durante as fases de análise e design, incluindo o modelo de domínio, os casos de uso, a arquitetura da aplicação e os diagramas que suportam a estrutura e organização do sistema. Estes elementos permitem compreender a forma como os diferentes componentes da aplicação se relacionam e colaboram para satisfazer os requisitos definidos.

2.1 Modelo de Domínio

O modelo de domínio apresentado na Figura 1 representa os principais conceitos do sistema PescaTudo e as relações existentes entre eles. Este modelo permite identificar as entidades fundamentais da aplicação, nomeadamente utilizadores, clientes, fornecedores, produtos, encomendas, linhas de encomenda e avaliações, bem como as respetivas associações e responsabilidades no contexto da gestão de uma loja de pesca.

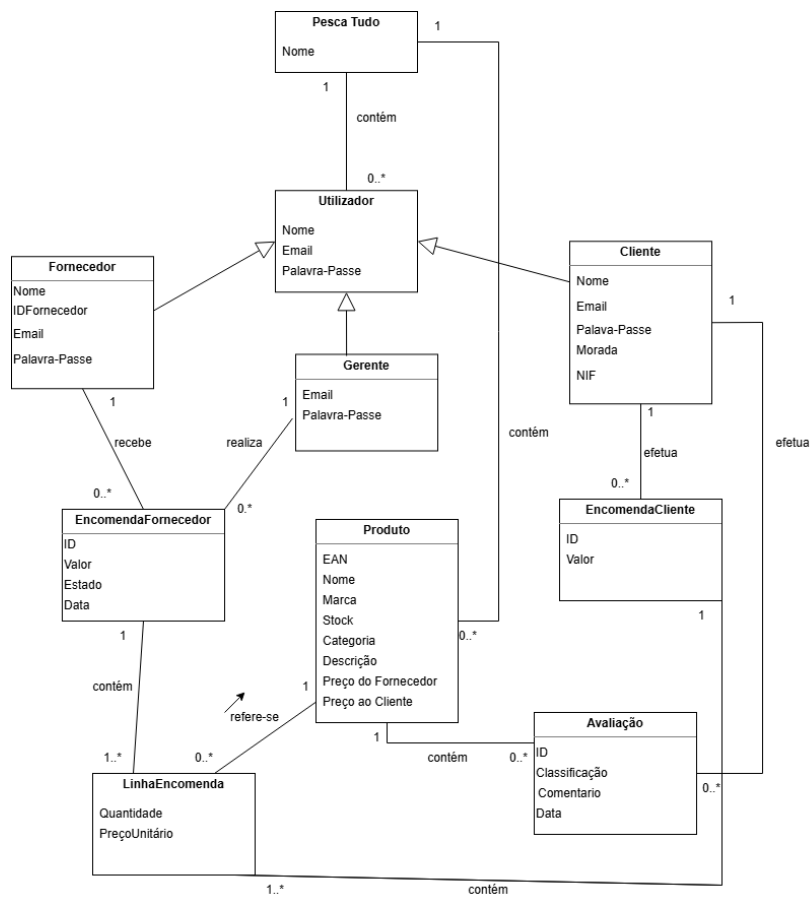


Figura 1 - Modelo de Domínio

2.2 Casos de Uso

Os diagramas de casos de uso apresentados nas figuras seguintes representam as principais funcionalidades disponibilizadas pela aplicação PescaTudo para os diferentes tipos de utilizadores do sistema. Estes diagramas permitem identificar as interações entre clientes, fornecedores e gerentes com a aplicação, evidenciando as operações que cada ator pode realizar de acordo com o seu perfil e nível de acesso.

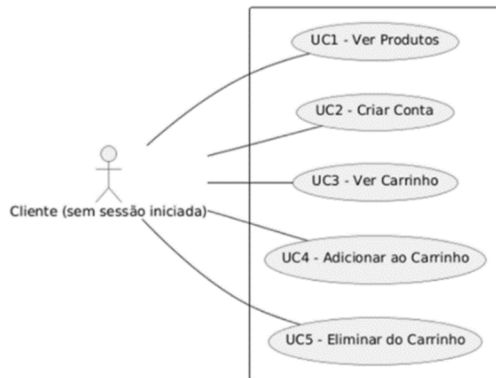


Figura 2 - Ucs cliente sem sessão iniciada

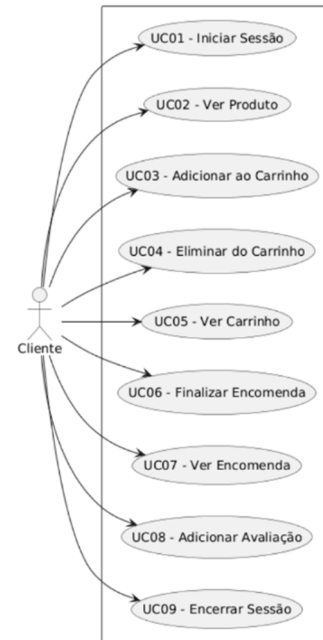


Figura 3 - UCs cliente

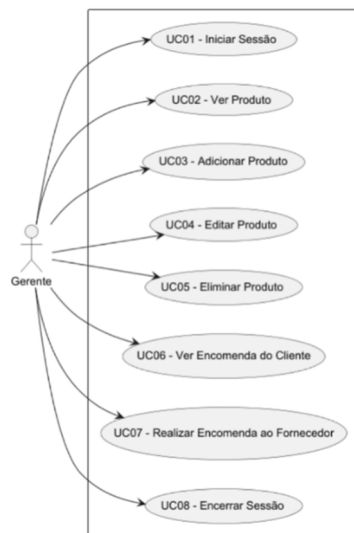


Figura 4 - Ucs gerente

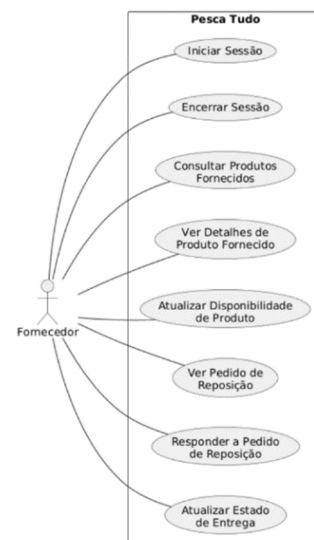


Figura 5 - Ucs fornecedor

2.3 Diagrama de Modelo de Classes

O diagrama de classes do modelo representa a estrutura principal da aplicação e as entidades que suportam a lógica de negócio do sistema.

Neste diagrama são apresentadas as classes associadas à gestão de produtos, clientes, fornecedores, encomendas, linhas de encomenda, avaliações e utilizadores, bem como os respetivos containers responsáveis por armazenar e organizar esses dados.

A classe principal, PescaTudo, agrega os diferentes containers da aplicação, permitindo centralizar o acesso às principais funcionalidades do sistema.

O diagrama inclui ainda as relações de herança, associação, composição e dependência entre as classes, bem como algumas exceções. Estas são utilizadas para validar dados, impedir duplicações, controlar autenticação e tratar situações em que não existem dados disponíveis.

O diagrama apresentado na Figura 6 permite compreender a organização interna do modelo e a forma como as principais entidades da aplicação se relacionam entre si.

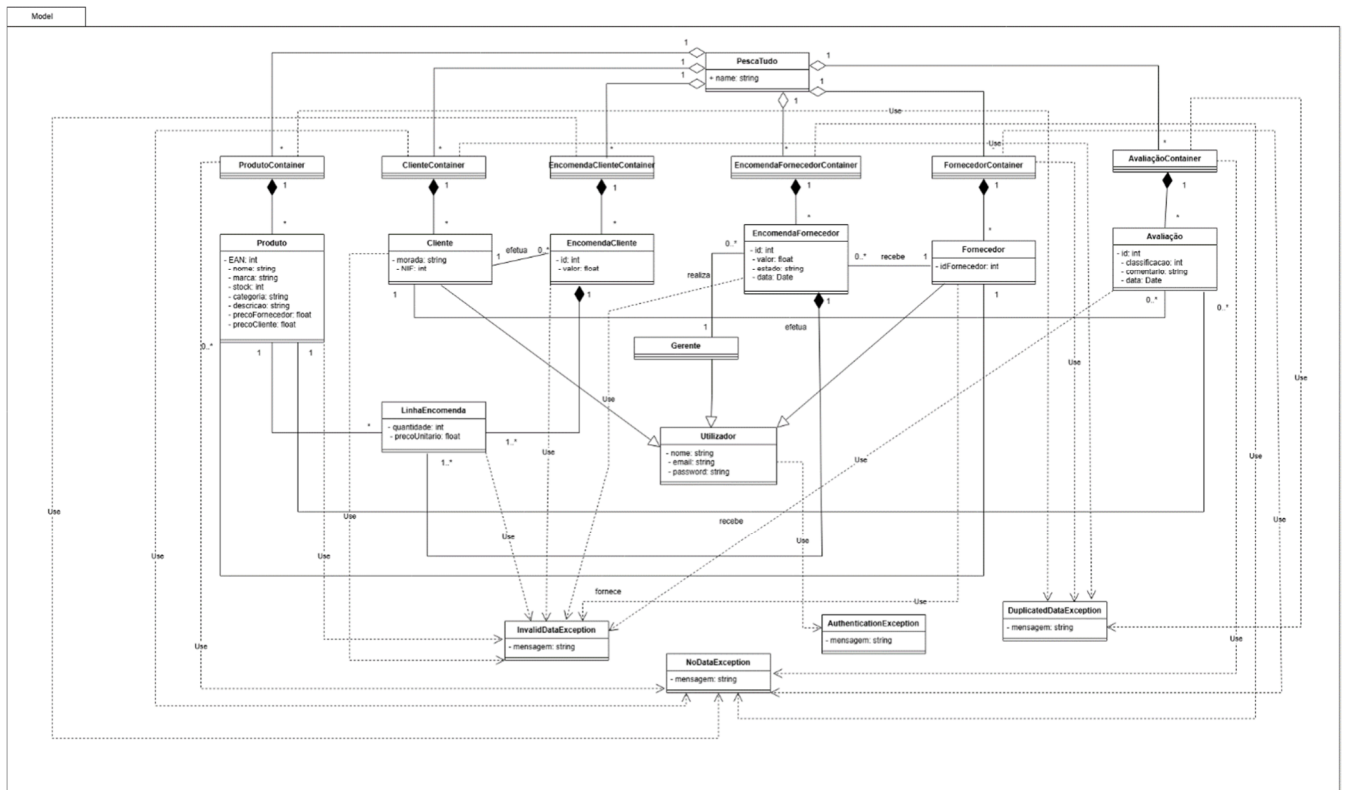


Figura 6 - Modelo de classes

2.4 Diagrama de classes de exceções

O diagrama de classes das exceções representa a estrutura utilizada pela aplicação para o tratamento de erros e situações excepcionais durante a execução do sistema.

Todas as exceções implementadas herdam da classe base `std::exception`, permitindo um mecanismo uniforme de gestão de erros em toda a aplicação.

As exceções definidas no sistema PescaTudo permitem tratar diferentes cenários, como introdução de dados inválidos, ausência de informação, duplicação de dados e falhas de autenticação.

O diagrama apresentado Figura 7 ilustra a hierarquia das exceções implementadas no sistema.

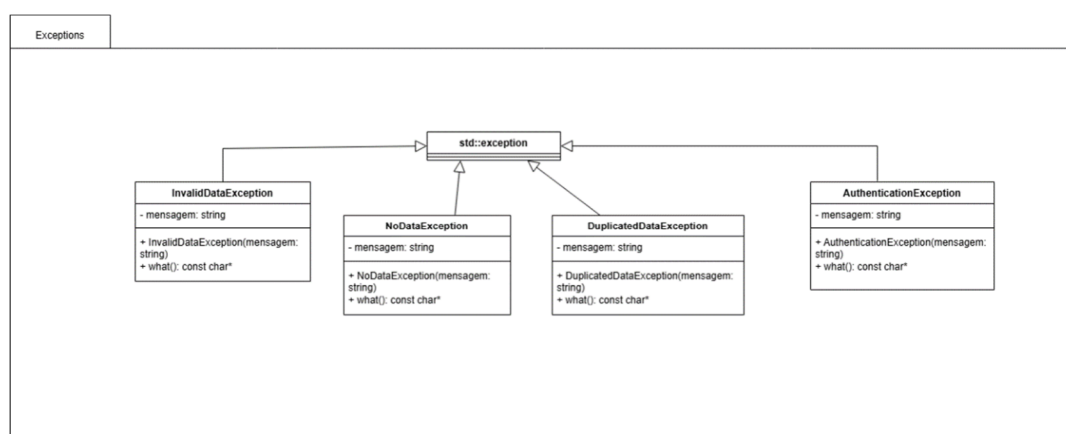


Figura 7 - Diagrama de exceções

3 Implementação

Após a definição dos requisitos e da estrutura da solução, procedeu-se à implementação da aplicação PescaTudo em linguagem C++. Nesta fase foram desenvolvidas as principais funcionalidades associadas aos diferentes perfis de utilizador do sistema, nomeadamente clientes, fornecedores e gerentes. As secções seguintes apresentam algumas das funcionalidades mais relevantes implementadas, destacando a sua lógica de funcionamento e a forma como contribuem para satisfazer os requisitos definidos para a aplicação.

3.1 Cliente: `viewProductsGuest()`

A função `viewProductsGuest()` permite apresentar a lista de produtos disponíveis na loja tanto a utilizadores sem sessão iniciada como a clientes autenticados. Inicialmente, a função obtém o conjunto de produtos armazenados no sistema e verifica se existem produtos disponíveis. De seguida, determina qual o carrinho a utilizar: caso exista uma

sessão ativa, é usado o carrinho associado ao cliente autenticado; caso contrário, é criado um carrinho temporário para o utilizador sem sessão. Após apresentar os produtos, a função disponibiliza opções para adicionar produtos ao carrinho, visualizar o carrinho ou regressar ao menu anterior. Esta implementação permite reutilizar a mesma lógica para diferentes perfis de acesso.

```
void Controller::viewProductsGuest() {
    const ProductContainer& products = store.getProducts();
    int option;

    Cart* cartPtr = nullptr;

    if (isAuthenticated()) {
        cartPtr = &clientCarts[loggedInClient->getEmail()];
    } else {
        static Cart guestCart;
        cartPtr = &guestCart;
    }

    Cart& cart = *cartPtr;

    std::cout << "\n--- Available Products ---\n";

    if (products.getSize() == 0) {
        std::cout << "No products available.\n";
        return;
    }

    for (int i = 0; i < products.getSize(); i++) {
        const Product& p = products.getProduct(i);

        std::cout << "ID: " << p.getId() << "\n";
        std::cout << "Name: " << p.getName() << "\n";
        std::cout << "Brand: " << p.getBrand() << "\n";
        std::cout << "Category: " << p.getCategory() << "\n";
        std::cout << "Description: " << p.getDescription() << "\n";
        std::cout << "Price: " << std::fixed << std::setprecision(2)
                    << p.getPriceClient() << " EUR\n";
        std::cout << "Stock: " << p.getStock() << "\n";
        std::cout << "-----\n";
    }

    do {
        std::cout << "1. Add to cart\n";
        std::cout << "2. View cart\n";
        std::cout << "0. Back to menu\n";
        std::cout << "Option: ";
        std::cin >> option;

        switch (option) {
            case 1:
                try {
                    addToCart(cart);
                } catch (const ProductNotFoundException& e) {
                    std::cout << "Error: " << e.what() << std::endl;
                }
                break;

            case 2:
                viewCart(cart);
                break;

            case 0:
                std::cout << "Returning to client menu...\n";
                break;

            default:
                std::cout << "Invalid option.\n";
        }
    } while (option != 0);
}
```

3.2 Cliente: completeOrder()

A função `completeOrder()` é responsável por finalizar uma encomenda efetuada por um cliente com sessão iniciada. Inicialmente, a função verifica se o carrinho contém produtos, envia uma exceção caso esteja vazio. De seguida, valida se existe uma sessão ativa e confirma a disponibilidade de stock para todos os produtos presentes no carrinho. Após a validação, as quantidades correspondentes são descontadas do stock disponível e é criada uma nova encomenda associada ao cliente autenticado. Por fim, a encomenda é registada no histórico do cliente e o carrinho é limpo, e a encomenda é concluída com sucesso.

```
void Controller::completeOrder(Cart& cart) {
    if (cart.isEmpty()) {
        throw EmptyCartException();
    }
    if (!isAuthenticated()) {
        std::cout << "You must be logged in to complete the order.\n";
        return;
    }
    // Subtrair stock
    for (const auto& item : cart.getItems()) {
        Product& product = store.findProductById(item.first.getId());
        if (item.second > product.getStock()) {
            std::cout << "Error: Product " << product.getName() << "
doesn't have enough stock.\n";
            return;
        }
        product.reduceStock(item.second);
    }
    // Criar e guardar a encomenda no cliente autenticado
    ClientOrder order(cart.getItems(), cart.getTotal());
    loggedInClient->addOrder(order);
    std::cout << "Order completed successfully!\n";
    cart.clear(); // Limpa o carrinho
}
```

3.3 Cliente: evaluateProduct()

A função `evaluateProduct()` permite a um cliente autenticado adicionar uma avaliação a um produto existente na loja. Inicialmente, a função verifica se existe uma sessão iniciada e apresenta a lista de produtos disponíveis. De seguida, o utilizador seleciona o produto que pretende avaliar e introduz uma classificação entre 1 e 5 estrelas, bem como um comentário associado. Após validar os dados introduzidos, é criada uma nova avaliação e esta é armazenada no sistema através do respetivo container de avaliações. Esta funcionalidade permite recolher a opinião dos clientes relativamente aos produtos disponibilizados pela loja.

```
void Controller::evaluateProduct() {
    if (!isAuthenticated()) {
        std::cout << "You must be logged in to evaluate a product.\n";
        return;
    }

    int productId;
    int stars;
    std::string comment;
```



```

std::cout << "\n--- Evaluate Product ---\n";

listProducts();

std::cout << "Enter product ID: ";
std::cin >> productId;

bool productExists = false;

ProductContainer& products = store.getProducts();

for (int i = 0; i < products.getSize(); i++) {
    if (products.getProduct(i).getId() == productId) {
        productExists = true;
        break;
    }
}

if (!productExists) {
    std::cout << "Product not found.\n";
    return;
}

std::cout << "Stars (1 to 5): ";
std::cin >> stars;

if (stars < 1 || stars > 5) {
    std::cout << "Invalid number of stars. Use a value between 1 and 5.\n";
    return;
}

std::cin.ignore();

std::cout << "Comment: ";
std::getline(std::cin, comment);

int newId = store.getEvaluations().getNextId();

Evaluation evaluation(
    newId,
    productId,
    loggedInClient->getEmail(),
    stars,
    comment
);

store.addEvaluation(evaluation);

std::cout << "Evaluation added successfully!\n";
}

```

3.4 Gerente: addProduct()

A função `addProduct()` permite ao gerente adicionar novos produtos ao catálogo da loja. Para tal, são recolhidas informações como o nome, marca, quantidade em stock, categoria, descrição e preços de compra e venda do produto. Após a introdução dos dados, a função determina automaticamente um identificador único para o novo produto e procede à sua criação e armazenamento no sistema. Por fim, é apresentada uma mensagem de confirmação e a lista atualizada de produtos disponíveis na loja.

```

void Controller::addProduct() {
    std::cin.ignore();

    std::string name, brand, category, description;
    int stock;
    float priceSupplier, priceClient;

```

```

std::cout << "\n--- Add New Product ---\n";

std::cout << "Name: ";
std::getline(std::cin, name);

std::cout << "Brand: ";
std::getline(std::cin, brand);

std::cout << "Stock quantity: ";
std::cin >> stock;
std::cin.ignore();

std::cout << "Category: ";
std::getline(std::cin, category);

std::cout << "Description: ";
std::getline(std::cin, description);

std::cout << "Supplier price (EUR): ";
std::cin >> priceSupplier;

std::cout << "Selling price (EUR): ";
std::cin >> priceClient;

ProductContainer& products = store.getProducts();

int newId = 1;

for (int i = 0; i < products.getSize(); i++) {
    const Product& product = products.getProduct(i);

    if (product.getId() >= newId) {
        newId = product.getId() + 1;
    }
}

Product newProduct(newId, name, brand, stock, category, description,
priceSupplier, priceClient);
products.addProduct(newProduct);

std::cout << "Product added successfully!\n";

listProducts();
}

```

3.5 Fornecedor: ListSupplierRestockRequests()

A função `listSupplierRestockRequests()` é responsável por apresentar os pedidos de reposição associados ao fornecedor autenticado. Inicialmente, verifica se existe uma sessão de fornecedor ativa, impedindo o acesso caso a autenticação não tenha sido realizada. Em seguida, obtém o contentor de encomendas a fornecedores e percorre todos os pedidos existentes, selecionando apenas aqueles cujo identificador do fornecedor corresponde ao utilizador autenticado. Para cada pedido encontrado, são apresentados o número da encomenda, a data e o respetivo estado. Caso não existam pedidos de reposição associados ao fornecedor, é apresentada uma mensagem informativa ao utilizador.

```

void Controller::listSupplierRestockRequests() {
    if (!isSupplierAuthenticated()) {
        std::cout << "Access denied. Please login first.\n";
        return;
    }

    const SupplierOrderContainer& orders = store.getSupplierOrders();
    bool found = false;
}

```

```

std::cout << "\n--- Restock Requests for "
          << loggedInSupplier->getName() << " ---\n";

for (int i = 0; i < orders.getSize(); i++) {
    const SupplierOrder& order = orders.getOrder(i);

    if (order.getSupplier().getId() != loggedInSupplier->getId()) {
        continue;
    }

    found = true;
    std::cout << "Order #" << order.getOrderNumber()
              << " | Date: " << order.getDate()
              << " | Status: " << order.getStatusText()
              << "\n";
}

if (!found) {
    std::cout << "No restock orders.\n";
}
}

```

3.6 Fornecedor: respondToSupplierRestockRequest()

A função `respondToSupplierRestockRequest()` permite ao fornecedor autenticado responder a um pedido de reposição. Inicialmente, verifica se existe uma sessão ativa e solicita o número da encomenda pretendida. Em seguida, procura o pedido no contentor de encomendas, confirmando se este pertence ao fornecedor autenticado e se ainda se encontra pendente. Caso alguma destas condições não seja cumprida, a operação é interrompida e é apresentada uma mensagem de erro. Após a validação, são exibidos os detalhes do pedido e o fornecedor pode selecionar uma confirmação total, uma confirmação parcial ou a rejeição da encomenda. Por fim, o estado do pedido é atualizado de acordo com a resposta escolhida e o sistema apresenta uma mensagem de sucesso com o novo estado registado.

```

void Controller::respondToSupplierRestockRequest() {
    if (!isSupplierAuthenticated()) {
        std::cout << "Access denied. Please login first.\n";
        return;
    }

    int orderNumber;
    std::cout << "Enter order number: ";
    std::cin >> orderNumber;

    SupplierOrderContainer& orders = store.getSupplierOrders();
    SupplierOrder* order = findOwnedSupplierOrder(
        orders,
        orderNumber,
        loggedInSupplier->getId()
    );

    if (order == nullptr || !order->isPending()) {
        std::cout << "Order cannot be answered. It may not exist, may
        belong "
                  "to another supplier, or may already have a
        response.\n";
        return;
    }

    showSupplierOrderDetails(*order);

    std::cout << "\n1. Total confirmation\n";
    std::cout << "2. Partial confirmation\n";
    std::cout << "3. Reject\n";
    std::cout << "Response: ";

    int response;
}

```

```

std::cin >> response;

switch (response) {
    case 1:
        order->setOrderStatus(SupplierOrderStatus::Confirmed);
        break;

    case 2:
        order-
>setOrderStatus(SupplierOrderStatus::PartiallyConfirmed);
        break;

    case 3:
        order->setOrderStatus(SupplierOrderStatus::Rejected);
        break;

    default:
        std::cout << "Invalid response.\n";
        return;
}

std::cout << "Restock request response registered successfully. "
        << "New status: " << order->getStatusText() << "\n";
}

```

4 Testes

A fase de testes é fundamental para garantir a qualidade, a robustez e o correto funcionamento da aplicação Pesca Tudo. De acordo com os requisitos definidos, a estratégia de testes implementada dividiu-se em duas vertentes principais: testes unitários e testes de integração. Os testes unitários focaram-se na validação isolada das classes do modelo de domínio, garantindo o correto funcionamento de todos os construtores, funções e operadores. Por outro lado, os testes de integração consistiram na execução da aplicação para validar todos os Casos de Uso (Use Cases) do sistema de ponta a ponta.

4.1 ClientTest

O primeiro exemplo demonstra os testes realizados à classe Client, focando-se na cobertura dos construtores e na validação dos métodos de negócio, utilizando rigorosamente o padrão Arrange-Act-Assert

```

#include <gtest/gtest.h>
#include "model/Client.h"
#include "model/ClientOrder.h"

// 1. Testar o Construtor por Omissão (Vazio)
TEST(ClientTest, DefaultConstructor) {
    // Arrange
    Client c;

    // Act
    std::string email = c.getEmail();
    std::string password = c.getPassword();
    int ordersSize = c.getOrders().size();

    // Assert
    EXPECT_EQ(email, "");
    EXPECT_EQ(password, "");
    EXPECT_EQ(ordersSize, 0);
}

// 2. Testar o Construtor Parametrizado
TEST(ClientTest, ParameterizedConstructor) {
    // Arrange
    std::string expectedEmail = "joao@email.com";
    std::string expectedPassword = "password123";

    // Act
    Client c(email, expectedPassword);

    // Assert
    EXPECT_EQ(c.getEmail(), expectedEmail);
    EXPECT_EQ(c.getPassword(), expectedPassword);
    // Assumimos que o nome começa vazio, visto que não é passado no construtor
    EXPECT_EQ(c.getName(), "");
}

```

4.2 ProductContainerTest

O segundo exemplo ilustra os testes à classe ProductContainer. Neste caso, além da verificação de construtores (incluindo o construtor de cópia), é dada especial atenção ao tratamento de exceções personalizadas (como a ProductNotFoundException), simulando um cenário onde o sistema tenta aceder a um produto inexistente

```

#include <gtest/gtest.h>
#include "model/ProductContainer.h"
#include "exceptions/ProductNotFoundException.h"

// 1. Testar Construtor por Omissão
TEST(ProductContainerTest, DefaultConstructor) {
    // Arrange
    ProductContainer container;

    // Act
    int size = container.getSize();
    bool isEmpty = container.empty();

    // Assert
    EXPECT_EQ(size, 0);
    EXPECT_TRUE(isEmpty);
}

// 2. Testar Adição de Produto (Garantir que não lança exceção, estilo slide 6)
TEST(ProductContainerTest, AddValidProduct) {
    // Arrange
    ProductContainer container;
    Product p(1, "Anzol", "MarcaX", 100, "Acessorios", "Desc", 0.5f, 1.0f);
    bool flag = true;

    // Act
    try {
        container.addProduct(p);
    } catch (...) {
        flag = false; // Se algo falhar na adição, a flag fica a false
    }

    // Assert
    EXPECT_TRUE(flag);
    EXPECT_EQ(container.getSize(), 1);
}

```

```
// 3. Testar Pesquisa de Produto Inexistente (Garantir que LANÇA a exceção correta)
TEST(ProductContainerTest, FindInvalidProductThrowsException) {
    // Arrange
    ProductContainer container;
    bool exceptionThrown = false; // Começamos a false porque esperamos que a exceção mude isto

    // Act
    try {
        // Tentamos procurar um ID que não existe num contentor vazio
        container.findById(1999);
    } catch (ProductNotFoundException& e) {
        exceptionThrown = true; // Sucesso: Apanhou a exceção certa!
    }

    // Assert
    EXPECT_TRUE(exceptionThrown);
}

// 4. Testar Remoção de Produto
TEST(ProductContainerTest, RemoveProduct) {
    // Arrange
    ProductContainer container;
    Product p(1, name: "Isco", brand: "MarcaY", stock: 50, category: "IscoS", description: "Desc", priceSupplier: 2.0f, priceClient: 4.0f);
    container.addProduct(p);

    // Act
    container.removeProduct(1);

    // Assert
    EXPECT_TRUE(container.empty());
    EXPECT_EQ(container.getSize(), 0);
}
}
```

```
// 5. Testar Construtor de Cópia (Regra dos 3)
TEST(ProductContainerTest, CopyConstructor) {
    // Arrange
    ProductContainer original;
    Product p(1, name: "Cana", brand: "Shimano", stock: 10, category: "Canas", description: "Desc", priceSupplier: 50.0f, priceClient: 100.0f);
    original.addProduct(p);

    // Act
    ProductContainer copy(original);

    // Assert
    EXPECT_EQ(copy.getSize(), 1);
    EXPECT_EQ(copy.findById(1).getName(), "Cana");
}

// 6. Testar Operador de Atribuição (operator=)
TEST(ProductContainerTest, AssignmentOperator) {
    // Arrange
    ProductContainer original;
    Product p(1, name: "Carreto", brand: "Daiwa", stock: 5, category: "Carretos", description: "Desc", priceSupplier: 40.0f, priceClient: 80.0f);
    original.addProduct(p);
    ProductContainer assigned;

    // Act
    assigned = original;

    // Assert
    EXPECT_EQ(assigned.getSize(), 1);
    EXPECT_EQ(assigned.findById(1).getName(), "Carreto");
}
}
```

4.2 Testes de Integração

Por forma a assegurar que as várias componentes do sistema interagem corretamente, a aplicação foi executada na sua totalidade para a validação dos fluxos reais dos utilizadores. Todos os Casos de Uso (UC) identificados na fase de análise foram testados de ponta a ponta.

Use Case	Percentagem
UC01 - Iniciar Sessão	100%
UC02 - Ver Produto	100%
UC03 - Adicionar ao Carrinho	100%
UC04 - Eliminar do Carrinho	100%
UC05 - Ver Carrinho	100%
UC06 - Finalizar Encomenda	100%
UC07 - Ver Encomenda	100%
UC08 - Adicionar Avaliação	100%
UC09 - Encerrar Sessão	100%
Gestão (Fornecedor / Gerente)	100%

5 Conclusão

Este projeto permitiu a implementação das principais funcionalidades definidas para a aplicação PescaTudo, consolidou o trabalho de análise e design desenvolvido nas fases anteriores. Durante esta etapa foi possível implementar funcionalidades associadas aos diferentes perfis de utilizador do sistema, nomeadamente clientes, fornecedores e gerentes, o que garantiu o suporte às operações fundamentais de gestão da loja.

Para além da implementação das funcionalidades previstas, foi desenvolvido mecanismos de validação e tratamento de exceções, contribuiu para a redução de erros e o correto funcionamento da aplicação. Adicionalmente, foi realizado testes unitários às principais classes do sistema, permitindo verificar o correto funcionamento dos componentes implementados e assegurar a consistência dos dados manipulados pela aplicação.

Os resultados obtidos demonstram que os objetivos inicialmente estabelecidos para o projeto foram alcançados com sucesso, tendo sido desenvolvida uma solução funcional capaz de suportar a gestão de produtos, encomendas, fornecedores, clientes e avaliações. A aplicação apresenta uma estrutura organizada, baseada nos princípios abordados na unidade curricular, constituindo uma solução coerente e adequada aos requisitos propostos para o projeto PescaTudo.